

Gasverbrauch-Statistik in ioBroker

Tages-, Wochen- und Monatswerte aus dem Zählerstand des HmIP-ESI-GAS

Datei	gasverbrauch_statistik.js
Version	2.0
Stand	20.09.2026
Adapter	javascript (Typ Javascript/ECMAScript)
Erstellt für	ssbingo
Benötigt für	ioBroker.vis-2-widgets-wolf (Gaszähler-Widget)

1. Ausgangslage

Der **HmIP-ESI-GAS** zeigt in der CCU-WebUI vier Statistikwerte an – *Heute, Gestern, Vergangene 7 Tage, Vergangene 30 Tage*. Diese Werte sind **keine Gerätedatenpunkte**: Sie werden CCU-intern aus den gespeicherten Zählerständen berechnet und liegen als Metadaten vor. Über `hm-rpc` kommen sie deshalb grundsätzlich nicht in ioBroker an.

In ioBroker verfügbar sind:

Quelle	Datenpunkt	Inhalt
hm-rpc	GAS_FLOW , GAS_FLOW_STATUS	aktueller Gasfluss in m³/h
hm-rpc	GAS_VOLUME , GAS_VOLUME_STATUS	Volumen in m³
hm-rega	svEnergyCounter...<Seriennummer>	Zählerstand – versteckte Systemvariable

Der Zählerstand ist monoton steigend und damit die einzige Größe, die man wirklich braucht: Alles andere lässt sich daraus ableiten. Genau das macht dieses Skript – unabhängig von CCU-Firmware-Internia, die HomeMatic jederzeit ändern kann.

Hinweis zur versteckten Systemvariablen: Damit `svEnergyCounter...` in ioBroker erscheint, muss im Adapter **hm-rega** unter *Synchronisieren* die Option für **nicht sichtbare Variablen** aktiviert werden. Danach die Instanz neu starten und im Objektbaum nach der Seriennummer des Geräts filtern.

2. Installation

1. Im Adapter **javascript** ein neues Skript vom Typ *Javascript/ECMAScript* anlegen.
2. Inhalt von `gasverbrauch_statistik.js` einfügen.
3. Im Konfigurationsblock oben mindestens `SRC` anpassen (siehe Abschnitt 3).

4. Skript starten und das Log prüfen – die States werden automatisch angelegt.

Voraussetzungen:

- Adapter `javascript` (JavaScript-Engine, Node ≥ 18)
- Objektbaum `0_userdata.0` (Standard; anderer Pfad über `PFAD` möglich)
- optional: ein History-Adapter für die einmalige Vorbefüllung (Abschnitt 5)

3. Konfiguration

Alle Einstellungen stehen im Kopf des Skripts.

Konstante	Default	Bedeutung
<code>SRC</code>	<code>'hm-rega.0.12345'</code>	Pflichtangabe. State mit dem Zählerstand in m ³ , z. B. die Rega-Variable <code>svEnergyCounter...</code> oder <code>hm-rpc.0.<SERIAL>.2.GAS_VOLUME</code>
<code>PFAD</code>	<code>'0_userdata.0.Gas'</code>	Zielordner für die erzeugten States
<code>INKL_HEUTE</code>	<code>false</code>	<code>false</code> = 7-/30-Tage-Summe zählt nur abgeschlossene Tage (Verhalten der CCU). <code>true</code> = laufender Tag wird mitgezählt
<code>TAGE_HISTORIE</code>	<code>70</code>	Anzahl vorgehaltener Tageswerte. Für <i>Letzter Monat</i> sind mindestens 62 nötig
<code>NK</code>	<code>3</code>	Nachkommastellen der Verbrauchswerte
<code>HISTORY_INSTANCE</code>	<code>'influxdb.0'</code>	Instanz für die Vorbefüllung. <code>''</code> schaltet die Funktion ab. Auch <code>'history.0'</code> oder <code>'sql.0'</code>
<code>BACKFILL_BEIM_START</code>	<code>true</code>	Beim allerersten Start automatisch aus der History vorbefüllen
<code>DEBUG</code>	<code>false</code>	Zusätzliche Log-Ausgaben bei jeder Berechnung

4. Erzeugte States

Alle unterhalb von `PFAD` (Default `0_userdata.0.Gas`):

State	Typ	Rolle	Inhalt
<code>Zaehlerstand</code>	number	<code>value.volume</code>	gespiegelter Zählerstand in m ³
<code>Heute</code>	number	<code>value</code>	Verbrauch seit Mitternacht
<code>Gestern</code>	number	<code>value</code>	Verbrauch des Vortags
<code>Letzte7Tage</code>	number	<code>value</code>	Summe der letzten 7 Tage
<code>Letzte30Tage</code>	number	<code>value</code>	Summe der letzten 30 Tage

State	Typ	Rolle	Inhalt
DieserMonat	number	value	Summe seit Monatsbeginn inkl. heute
LetzterMonat	number	value	Summe des Vormonats
Basis	number	value.volume	Zählerstand um Mitternacht (intern)
BasisDatum	string	date	Datum dieser Basis, YYYY-MM-DD (intern)
Historie	string	json	Tageswerte als JSON-Ringpuffer (intern)
Backfill	boolean	button	löst die Vorbefüllung aus (Abschnitt 5)

Die drei internen States nicht von Hand ändern – sie bilden den Zustand ab, aus dem alle anderen Werte berechnet werden.

Format der Historie:

```
[{"d": "2026-09-18", "v": 1.3}, {"d": "2026-09-19", "v": 1.6}]
```

Ein Eintrag je **abgeschlossenem** Tag. Der laufende Tag steht nicht darin, sondern ergibt sich aus `Zaehlerstand - Basis`.

5. Vorbefüllung aus dem History-Adapter

Ohne Vorbefüllung sind die 7-/30-Tage-Werte erst nach 7 bzw. 30 Tagen Laufzeit vollständig. Wird der Zählerstand bereits in `influxdb`, `history` oder `sql` aufgezeichnet, baut das Skript die Historie daraus in einem Rutsch auf.

Automatisch: beim allerersten Start, sofern `HISTORY_INSTANCE` gesetzt und `BACKFILL_BEIM_START = true` ist.

Manuell: den State `<PFAD>.Backfill` auf `true` setzen (Objektbaum oder VIS-Button). Die Historie wird dabei komplett neu aufgebaut.

Verfahren: Für jede Tagesgrenze (00:00 Uhr) wird der Zählerstand per `getHistory` mit `aggregate: 'max'` über ein Fenster davor abgefragt – beim monoton steigenden Zähler ist das Maximum des Fensters genau der Stand zur Tagesgrenze. Der Tagesverbrauch ist die Differenz zweier aufeinanderfolgender Tagesgrenzen. Pro Tagesgrenze fällt nur eine Abfrage mit einem Rückgabewert an; auch bei minütlichem Logging bleibt die Last gering. Liefert das erste Fenster (2 Tage) nichts, wird auf 7 und dann 30 Tage erweitert.

Tage, für die keine History-Daten vorliegen, werden ausgelassen statt mit falschen Werten gefüllt; das Log nennt die Anzahl. Liefert die History gar nichts, bricht die Vorbefüllung mit einer Warnung ab und der laufende Betrieb beginnt bei null – ein typischer Grund ist, dass der Quell-State im History-Adapter nicht zur Aufzeichnung aktiviert ist.

6. Funktionsweise im laufenden Betrieb



Auslöser der Berechnung

Auslöser	Zweck
Änderung von SRC	laufende Aktualisierung von Heute
<code>schedule('1 0 * * *')</code>	Tagesabschluss um 00:01 Uhr
<code>schedule('5 * * * *')</code>	stündliches Sicherheitsnetz
<code>Backfill = true</code>	Vorbefüllung aus der History

Überlappende Läufe sind durch eine Sperre ausgeschlossen – wichtig, weil die Vorbefüllung länger dauern kann als das Intervall des Zählers.

Robustheit

- **Verpasster Mitternachtslauf:** Der Tagesabschluss hängt nicht allein am Cron. Bei jeder Zähleränderung wird geprüft, ob sich das Datum gegenüber `BasisDatum` geändert hat, und der Abschluss gegebenenfalls nachgeholt.
- **Ausfalltage:** War ioBroker mehrere Tage aus, werden die fehlenden Tage mit `0` aufgefüllt, damit die Reihe nicht verrutscht. Der in dieser Zeit angefallene Verbrauch wird dem ersten Tag der Lücke zugeschlagen – die Summen über 7 und 30 Tage bleiben dadurch korrekt, die einzelnen Tageswerte innerhalb der Lücke nicht.
- **Zählerreset / Zählerwechsel:** Sinkt der Zählerstand, wird die Basis zurückgesetzt und der betroffene Tag mit `0` gewertet, statt negative Werte zu erzeugen. Eine Warnung landet im Log.
- **Ungültige Werte:** `null`, `undefined` und nicht-numerische Werte werden ignoriert, nicht als 0 verarbeitet.

7. Bekannte Grenzen

- **Tagesgrenze.** Die Basis wird mit dem zuletzt gemeldeten Zählerstand gesetzt. Meldet der ESI zuletzt um 23:40 Uhr, zählt der Verbrauch von 23:40 bis 00:00 Uhr zum Folgetag. Bei den üblichen Melderaten liegt der Fehler im Bereich weniger Hundertstel m³ und gleicht sich über die Tage aus.

- **Rollierende Fenster.** Letzte 7/30 Tage summieren die letzten 7 bzw. 30 abgeschlossenen Tage. Die CCU rechnet gleichartig, kann bei einer Neuanlage aber eine längere Vorgeschichte haben – kleine Abweichungen zur WebUI sind in den ersten Wochen normal.
- **Sommer-/Winterzeit.** Alle Datumsgrenzen laufen über lokale Zeit; der 25-Stunden- und der 23-Stunden-Tag werden korrekt als ein Tag gewertet.
- **Keine Rekonstruktion ohne History.** Ohne History-Adapter beginnt die Statistik bei null; die CCU-Werte lassen sich nicht rückwirkend importieren.

8. Fehlersuche

Symptom	Ursache / Abhilfe
Log: Quell-State "..." existiert nicht	SRC falsch. Objekt-ID im Objektbaum kopieren
Alle Werte bleiben 0	Quell-State liefert keinen Zahlenwert – Typ prüfen (<code>number</code> , nicht <code>string</code>)
Log: History lieferte keine verwertbaren Daten	Quell-State ist im History-Adapter nicht zur Aufzeichnung aktiviert, oder <code>HISTORY_INSTANCE</code> stimmt nicht
Gestern bleibt 0	Noch kein Tagesabschluss gelaufen – erst am Folgetag gefüllt (oder Vorbefüllung nutzen)
Werte weichen von der CCU ab	<code>INKL_HEUTE</code> prüfen; die CCU zählt den laufenden Tag in den 7-/30-Tage-Werten nicht mit
Log: Zählerstand ist gesunken	Zählerwechsel oder Reset der Rega-Variablen – einmalig normal, wiederholt ein Hinweis auf eine instabile Quelle
Historie soll neu aufgebaut werden	<code>Backfill</code> auf <code>true</code> setzen, oder <code>Historie</code> auf <code>[]</code> setzen und Skript neu starten

Für detaillierte Ausgaben `DEBUG = true` setzen; das Skript protokolliert dann bei jeder Berechnung Zählerstand und alle abgeleiteten Werte.

9. Verwendung in VIS-2

Die States sind mit Rolle und Einheit versehen und lassen sich direkt binden:

```
{0_userdata.0.Gas.Heute}      m³
{0_userdata.0.Gas.Gestern}    m³
{0_userdata.0.Gas.Letzte7Tage} m³
{0_userdata.0.Gas.Letzte30Tage} m³
```

Für ein Balkendiagramm der letzten Tage eignet sich der State `Historie` (`role: json`) als Datenquelle – er enthält Datum und Verbrauch je Tag bereits in der Struktur, die die meisten Chart-Widgets erwarten.

10. Quellen

- HmIP-ESI Gas – nicht alle Werte im ioBroker: <https://forum.iobroker.net/topic/82859/hmip-esi-gas-nicht-alle-werte-im-iobroker>
- HmIP-ESI (ioBroker-Forum): <https://forum.iobroker.net/topic/73449/hmip-esi>
- HowTo – erweiterte CCU3/RM Verbrauchszähler in eigene SysVars schreiben: <https://homematic-forum.de/forum/viewtopic.php?t=82352>
- HmIP-ESI Datenpunkte für historische Daten: <https://homematic-forum.de/forum/viewtopic.php?t=81510>
- Metadaten per Script abfragen: <https://homematic-forum.de/forum/viewtopic.php?t=82029>